

Variational Neurons and Bayesian Neural Networks: Distinct Probabilistic Objects, Inference Locations, and Computational Granularity

Yves Ruffenach^[0009–0009–4737–0555]

Conservatoire National des Arts et Métiers, Strasbourg, France
yves@ruffenach.net

Abstract. Neural networks can represent uncertainty in different places. A Bayesian neural network (BNN) treats the model itself as uncertain after observing the training data. A heteroscedastic regressor—a model whose predicted noise level can vary with the input—can keep its hidden computation deterministic while predicting both a mean and an input-dependent noise level. EVE (Elemental Variational Expanse) uses a third arrangement: each declared unit can contain a small input-dependent latent state, meaning an internal random variable that is not directly observed, and that state participates in the forward computation. Earlier work introduced EVE; the experiments here characterize how its local state contributes to computation. Three controlled experiments are reported. First, while keeping every learned weight fixed, we deliberately give an input the local state associated with another input. This worsens point prediction: mean-squared error (MSE) increases by 0.0994 (95% confidence interval, CI, [0.0631, 0.136]). If only the input-dependent variance is reassigned across inputs, the MSE difference remains close to zero, but negative log-likelihood (NLL), a probabilistic-fit measure, worsens by 0.0842 ([0.0297, 0.139]). Second, when the high-noise region of the data-generating process is moved, the learned local variance moves with it. A tracking slope of 1 would mean one-for-one movement; the observed mean slope is 1.13 ([0.975, 1.29]). Third, in 50 fresh paired runs using different random seeds (controlled sources of experimental randomness), EVE is compared with a heteroscedastic output model having exactly the same 2,472 trainable parameters. The 95% confidence intervals for all four primary predictive measures—point error (MSE), probabilistic fit (NLL), full-distribution quality (CRPS), and calibration (agreement between predicted uncertainty and observed frequencies)—span zero, so this experiment detects no difference on those measures. The evidence shows that the local latent state is used by the trained computation and responds to controlled observation noise. In the matched comparison, predictive behavior is closely aligned with the simpler output-level comparator across the four primary measures. Together, these results distinguish uncertainty over a model, uncertainty at an output, and an input-dependent random state inside a neural unit.

Keywords: Bayesian neural networks · variational inference · variational neurons · local latent variables · EVE · computational granularity · uncertainty quantification

1 Introduction

Many neural networks are called *probabilistic*, but probability can enter a network in very different ways. The simplest way to see the distinction is to ask: *what quantity is random, and where in the computation is that random quantity used?*

A Bayesian neural network (BNN) usually treats the model itself as uncertain: after observing the training set, several weight configurations or functions remain plausible. A heteroscedastic regressor—that is, a model whose predicted noise level can change with the input—does something different. Its hidden computation may be fully deterministic, but its output predicts both a central value and an input-dependent noise level. EVE (Elemental Variational Expanse) uses a third arrangement: a hidden unit contains an input-dependent random latent state, and a sample from that state participates in the forward computation. Here, *latent state* simply means an internal random variable that is not directly observed. *Computational granularity* refers to the scale at which such a random variable is attached—for example, to the whole model, a representation, one unit, or the output. Here, *variational* means that the unit learns the parameters of a probability distribution and is trained with a criterion that includes a KL comparison to a reference distribution, rather than learning only one deterministic state. Bayesian-posterior semantics additionally require a declared probabilistic model and posterior target.

These constructions may use similar mathematical tools—for example Gaussian distributions, KL-based regularization, or repeated random sampling. A KL penalty can be viewed here as a cost for moving a learned distribution away from a declared reference distribution. The same mathematical tools can therefore describe uncertainty attached to different quantities [17,20]. This matters because an uncertainty estimate can only be interpreted correctly if the reader knows what is uncertain and where that uncertainty enters the computation.

Earlier work introduced the local EVE variational unit, studied its dimensionality and internal activity, examined readout from its latent state, and evaluated related variants in neural networks and language models [41,42,68,69,70,71,72,73,74,75]. Section 3 and Appendix A specify the boundary between those foundations and the experiments reported here.

Three focused questions organize the experiments:

1. **Does the trained network actually use the local latent state?** We change that state after training while leaving every learned weight unchanged.
2. **Does the learned local variance respond to a known change in observation noise?** We move the high-noise region of a synthetic problem while leaving its mean function unchanged.

3. **How does unit-local Gaussian variability compare with output-level uncertainty?** We compare EVE with a conventional output-uncertainty model that has exactly the same 2,472 trainable parameters and the same Gaussian predictive family on 50 fresh paired runs.

The results separate these roles clearly. The complete local state is used by the trained model: assigning an input the latent state of another input strongly degrades prediction. The input-dependent variance has a more specific role: disrupting only that variance worsens probabilistic scores while leaving the point-prediction error unresolved around zero. The learned local variance also follows a controlled relocation of observation noise. In the exactly matched 50-seed comparison, all four primary paired intervals span zero. The contribution is therefore a mechanistic and comparative characterization of the local state together with a tightly matched predictive comparison.

2 What Is Random, and Where?

The central distinction can be understood without the notation: **a BNN makes the model uncertain; a heteroscedastic predictor makes the output noise input-dependent; EVE makes an internal state of a declared unit input-dependent and random.** The equations below formalize this distinction.

A *random object* is the quantity to which a probability distribution is attached. A compact description of a probabilistic architecture should answer four questions:

1. What is random: weights, a latent representation, a local hidden state, a gate or mask, or an output variable?
2. What information determines its distribution: the full training dataset, the current input, a hidden representation, or some other context?
3. Where does this random quantity enter the forward computation?
4. What is sampled, averaged, integrated, or summarized to form the final prediction?

These questions separate *where sampling happens* from *what the model treats as uncertain*. For example, the local reparameterization trick can sample an intermediate pre-activation even though the inferential object remains a distribution over weights [26].

2.1 Bayesian neural networks: uncertainty over the model

Let w denote the network parameters and $\mathcal{D}$ the training dataset. A canonical BNN starts with a prior distribution over w and updates it after observing $\mathcal{D}$. In compact form,

$$p(w \mid \mathcal{D}) \propto p(\mathcal{D} \mid w)p(w), \quad p(y_* \mid x_*, \mathcal{D}) = \int p(y_* \mid x_*, w)p(w \mid \mathcal{D}) dw. \quad (1)$$

Table 1. The same broad probabilistic vocabulary can refer to different random quantities. The table states the distinction in operational terms.

Construction	What is random?	Where / conditioned on what?	How prediction is formed
Bayesian neural network	weights or functions	posterior conditioned on dataset $\mathcal{D}$	average over plausible models or functions
Variational latent model	latent variable z	commonly conditioned on an input or representation specified by the randomized computation	sample or average over the latent
Stochastic operator	mask, noise, gate, or path	input-dependent scale predicted at the output conditioned on the input to a declared hidden unit	aggregate realizations when required
Heteroscedastic predictor	output observation variable		use the predicted output density
Variational neuron	local latent z_i		use z_i inside the unit, then continue the network computation

The first expression says that training data change which weights are plausible. The second says that prediction averages over those plausible weights rather than committing to one point estimate. The posterior may be approximated by variational inference, Laplace methods, MCMC, or other techniques [33,47,40,11]. Thus *Bayesian* describes the probabilistic model and its conditioning on data; *variational* describes one possible method for approximating the resulting inference problem.

2.2 Variational neurons: an internal random state

Let h_i be the deterministic information arriving at unit i . EVE constructs a local Gaussian distribution whose mean and variance depend on that current hidden input:

$$q_{\phi_i}(z_i \mid h_i, c_i) = \mathcal{N}(\mu_{\phi_i}(h_i, c_i), \text{Diag}(\sigma_{\phi_i}^2(h_i, c_i))). \quad (2)$$

Here z_i is the local latent state, μ is its learned center, σ its learned scale, and c_i denotes any additional declared context. A sample can be written as

$$z_i = \mu_{\phi_i}(h_i, c_i) + \sigma_{\phi_i}(h_i, c_i) \odot \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, I), \quad (3)$$

and that sample is used inside the unit,

$$a_i = F_{\eta_i}(h_i, z_i). \quad (4)$$

In words, the same input h_i determines a distribution over possible local states, one state is sampled, and the unit uses it to produce its activation a_i .

Training also keeps the learned local distribution near a declared reference distribution p_{ψ_i} . For a set of units $\mathcal{I}$, the objective may be written

$$\mathcal{J} = \mathbb{E}_{\epsilon}[\ell_{\text{task}}(f(x; z), y)] + \sum_{i \in \mathcal{I}} \beta_i \text{KL}(q_{\phi_i}(z_i \mid h_i, c_i) \parallel p_{\psi_i}(z_i \mid c_i)) + \Omega. \quad (5)$$

The first term trains the network for its task. The KL term is a regularizer: it penalizes large departures of the learned local distribution from its reference. Ω denotes any additional declared regularization.

Definition 1 (Variational neuron). *A hidden unit is called a variational neuron when (i) it contains a declared input-conditioned latent distribution $q_{\phi_i}(z_i | h_i, c_i)$; (ii) the sampled or summarized state z_i is used by a unit-local forward map before downstream mixing; (iii) the latent has a declared reference or probabilistic target and a separately computable variational contribution; and (iv) the unit boundary and latent dimensionality are specified as part of the architecture before analysis.*

In plain terms, a variational neuron contains a distribution that belongs to a declared unit and participates in that unit’s computation before information is mixed downstream.

This definition concerns *location and computation*. Under Eq. (5), a task loss plus a KL penalty to a reference distribution defines *unit-level variational regularization*. Bayesian-posterior semantics apply when a joint probabilistic model and explicit posterior target are declared. The experiments use point-estimated weights and the regularized regime, while weight-posterior BNNs use a different inferential object.

A stochastic hidden unit satisfies Definition 1 when its random computation is accompanied by the declared local input-conditioned distribution, reference term, and unit boundary. Dropout masks, injected Gaussian noise, and sampled gates illustrate stochastic computation with different specifications.

3 Relation to Prior Work

3.1 Established EVE foundations

Earlier EVE studies established the architecture and several associated diagnostics. The original EVE work introduced the local variational computational unit and its local KL regularization [41]. A subsequent study examined latent dimensionality, capacity, and temporal persistence [42]. Later preprints studied internal activity measurements, frozen-state readout, the broader distributional-neuron framing, network and Transformer feasibility, and language-model monitoring [68,69,70,71,72,73,74,75].

These earlier studies establish EVE itself, the local Gaussian latent, local KL diagnostics, latent dimensionality, and the general measurement of internal probabilistic activity. Appendix A provides a work-by-work boundary.

The experimental contribution consists of (i) interventions on an already trained model that separately disturb the complete local state and its input-dependent variance, (ii) a controlled experiment that moves a known source of observation noise and tests whether the learned local variance follows it, and (iii) a fresh 50-seed comparison against an exactly parameter-matched output heteroscedastic model. The terminology in Section 2 makes the compared random objects and computational locations explicit.

3.2 Neighboring probabilistic architectures

Several established methods are close in mathematics but differ in where their random variable lives. VAEs use input-conditioned latents at example or representation level [27,39]. The variational information bottleneck places a stochastic bottleneck in a learned representation [3]. BNNs with local reparameterization may sample local pre-activations while the inferred quantity remains a distribution over weights [26]. Heteroscedastic regressors predict an input-dependent observation variance at the output [25], and VIFO performs variational inference at the final-layer output interface [63]. Recent Gaussian-process neurons provide another neuron-level probabilistic construction, but they propagate Gaussian measures analytically through GP-based units rather than using EVE’s reparameterized local latent state [76]. The relevant distinction is which quantity is random and where it is used.

4 Controlled Empirical Tests

The experiments use a deliberately simple one-dimensional regression problem in which the true mean and noise level are known. The noise is *heteroscedastic*: its standard deviation depends on the input. This controlled setting is useful because the source of variation is known exactly, so the internal EVE quantities can be tested against it. The aim is mechanistic understanding under controlled conditions. Full architectures, seeds, optimization settings, per-seed results, and secondary controls are reported in Appendix B.

4.1 Experimental setting and how to read the metrics

For each random seed, independently generated training, validation, and test sets contain 2,500, 900, and 1,500 observations. A *seed* is one repeated experimental run with its own controlled randomization. The main mechanistic model is the nonlinear $k = 2$ EVE unit with 2,472 trainable parameters. Ten paired seeds are used for the intervention, dimensionality, and noise-relocation studies.

A separate Gaussian-affine $k = 1$ EVE configuration is used for the final output comparison. “Gaussian-affine” means that the local Gaussian state enters the prediction linearly, so the predictive Gaussian distribution can be computed exactly rather than estimated by Monte Carlo sampling. Its hyperparameter setting is selected on development seeds 40–59, locked, and then evaluated on fresh test seeds 400–449. The comparator has the same 2,472 trainable parameters and the same Gaussian predictive family.

Four primary metrics are used. MSE measures squared point-prediction error. Negative log-likelihood (NLL) evaluates how much probability the model assigns to the observed target. The continuous ranked probability score (CRPS) evaluates the full predictive distribution. Mean absolute coverage error (MACE) summarizes calibration across several prediction-interval levels. Lower is better for all four metrics. A 95% confidence interval for a paired difference summarizes its uncertainty across seeds; when that interval spans zero, the direction of the paired difference remains unresolved at that resolution.

Table 2. Selected ten-seed intervention effects relative to ordinary EVE sampling. Positive differences indicate worse performance. All learned weights are kept fixed.

Intervention	Δ MSE (95% CI)	Δ NLL (95% CI)
Shuffle complete latent state across inputs	0.0994 [0.0631, 0.136]	0.713 [0.506, 0.920]
Shuffle only input-conditioned latent variance	−0.000147 [−0.000340, 0.000046]	0.0842 [0.0297, 0.139]
Use local mean instead of sampling	0.000114 [−0.000530, 0.000759]	2.82 [2.67, 2.97]

4.2 Does the trained network use the local state?

Each nonlinear EVE model is trained once. We then reuse the exact same trained weights and change only how its latent state is supplied at test time. This is important: if the output changes, the change cannot be attributed to retraining or to a different set of learned parameters. A saved set of trained parameters is often called a *checkpoint*.

Two interventions are especially informative. In the *complete-state shuffle*, each input receives a latent state taken from another input, breaking the learned input–state pairing. In the *variance-only shuffle*, the central latent value remains associated with the input but the input-dependent variance is reassigned across inputs. Table 2 reports the corresponding paired effects.

The complete-state shuffle produces a large increase in both MSE and NLL. The trained predictor therefore depends on the input-specific local state. The variance-only shuffle gives a different pattern: its MSE interval includes zero, but NLL and CRPS worsen. On this benchmark, the input-dependent variance affects the predictive spread while the point-prediction difference remains unresolved around zero. This establishes a functional role in the trained computation; semantic interpretation is assessed separately through the controlled noise manipulation.

4.3 Does local variance follow controlled observation noise?

The next experiment asks whether the learned local variance moves when the known source of data noise moves. The conditional mean function is kept exactly the same. Only the location of the high-variance region is changed: left, center, or right. Within a seed, the same input values and the same standardized Gaussian noise draws are reused, so the intended noise-location change is isolated.

After training, we compare the known conditional standard deviation with the trace of EVE’s local covariance. For a diagonal Gaussian, this trace is simply the sum of the local latent variances and can be read as a scalar measure of total local spread. The correlations are 0.747 ([0.653, 0.842]), 0.982 ([0.975, 0.989]), and 0.793 ([0.667, 0.918]) for the left, center, and right noise locations. We also

Table 3. Fresh 50-seed exact-capacity comparison. Differences are Gaussian-affine EVE minus the heteroscedastic output model; lower is better for all four metrics.

Metric	Paired difference	95% CI
MSE	-0.0000046	$[-0.0000331, 0.0000240]$
NLL	-0.000098	$[-0.000494, 0.000298]$
CRPS	-0.0000095	$[-0.0000394, 0.0000204]$
MACE	-0.000031	$[-0.000443, 0.000381]$

measure how far the center of the learned variance profile moves when the true high-noise region is moved. A tracking slope of 1 would mean one-for-one movement: shifting the true noise center by a given amount would shift the learned variance center by the same amount. The observed mean per-seed slope is 1.13 ($[0.975, 1.29]$).

Because the manipulated quantity is known by construction, the result supports a concrete interpretation within this synthetic setting: EVE’s learned local variance is responsive to observation-noise structure. Its semantic scope is the controlled data-generating process used in this experiment.

4.4 How does local-state uncertainty compare with output uncertainty?

The third experiment compares predictive behavior when Gaussian variability is represented inside the unit versus at the output.

To make this comparison as direct as possible, the EVE model and the heteroscedastic output model each contain exactly 2,472 trainable parameters and each produces an analytic Gaussian predictive distribution. The final configurations are fixed before evaluation on the 50 fresh paired test seeds.

All four primary confidence intervals span zero. The corresponding two-sided Wilcoxon p -values are 0.848, 0.383, 0.466, and 0.947. At the attained resolution, the two parameterizations show closely matched predictive behavior on the four primary metrics. Equivalence would require a prespecified practically meaningful margin; the analysis therefore reports paired difference intervals rather than an equivalence test.

The matched comparison uses an EVE stochastic unit at the terminal regression stage with an identity readout (Appendix B). It therefore isolates computational location under closely matched parameter count and predictive family. Placement deeper inside a multi-stage network, where the local state can influence later computation, defines a distinct experimental setting.

4.5 Secondary latent-dimensionality check

A secondary experiment varies the number of local latent coordinates, $k \in \{1, 2, 4, 8\}$, while keeping parameter count approximately matched. Relative to

$k = 2$, the predictive confidence intervals include zero, whereas the KL participation ratio changes systematically; at $k = 8$, the paired change is $+3.25$ ($[2.57, 3.93]$). The participation ratio is a descriptive measure of how broadly the KL contribution is distributed across latent coordinates. Because latent dimensionality was established in earlier EVE work [42], this experiment serves as a consistency check alongside the new intervention and matched-comparison evidence.

5 Discussion

5.1 What does the local variational state add?

The experiments establish a concrete capability: the model contains an internal, input-dependent random state that can be measured and changed at a known computational location after training. Changing the complete state changes the network’s point prediction; changing only its variance changes probabilistic scoring. In the synthetic noise experiment, the learned local variance also moves when the true observation-noise pattern moves. These findings make the local state more than a passive regularizer: it is part of the trained forward computation and can be probed directly.

In the exact 50-seed comparison, the four primary paired effects are very small and every confidence interval spans zero. The demonstrated contribution is therefore *local observability and intervention*, together with predictive behavior closely matched to the output-level comparator in this controlled setting. Multi-stage tasks involving routing, memory, reasoning, or decisions provide a separate setting in which computational location can be evaluated further.

5.2 Relation to Bayesian uncertainty

The distinction from a BNN is straightforward. The network weights in these experiments are single learned point estimates. EVE’s local distribution is conditioned on the current hidden input and regularized toward a reference distribution. Its random object is therefore an input-conditioned local state rather than a dataset-conditioned posterior over model weights. A future architecture could combine both mechanisms—uncertain weights and local latent states—while retaining two distinct sources of variation.

Labels such as *aleatoric* and *epistemic* require explicit semantic identification. The controlled noise-relocation experiment supports an observation-noise interpretation within its known synthetic data-generating process.

5.3 Scope of the evidence

The evidence covers one synthetic one-dimensional regression family, ten seeds for the mechanistic studies, and one 50-seed exact-capacity follow-up. The matched output comparison uses a terminal EVE unit, while deeper

placement defines a separate architectural setting. The interventions establish dependence of the trained output on the local state at the declared unit level; coordinate-specific necessity and identifiability are separate questions. The noise-relocation experiment identifies responsiveness to one known variance manipulation. Confidence intervals that span zero are reported as unresolved paired differences; equivalence requires a dedicated prespecified equivalence margin.

6 Conclusion

A probabilistic neural network is easier to interpret once two questions are separated: *what is random, and where is it used?* A BNN typically places uncertainty over weights or functions. A heteroscedastic regressor places an input-dependent noise estimate at the output. EVE places an input-dependent random state inside a declared unit.

The controlled experiments show that this EVE state is used by the trained predictor: disrupting the complete state degrades point and probabilistic prediction, and disrupting only its variance changes probabilistic scores. When the location of observation noise is deliberately moved, the learned local variance moves with it. An exactly parameter-matched 50-seed comparison with a heteroscedastic output model yields paired intervals spanning zero for MSE, NLL, CRPS, and calibration. The evidence therefore supports a local, manipulable probabilistic state together with closely matched predictive behavior in the controlled comparison. This distinction between model-level uncertainty, output-level uncertainty, and unit-local variability is the central interpretive result.

Data and Code Availability Protocol and numerical results are reported in Section 4 and Appendix B. The accompanying supplementary archive contains executable notebooks for the fixed-weight latent interventions, latent-dimension sweep, paired statistics and calibration, controlled noise tracking, the exact-capacity Gaussian-affine comparison, and the single-neuron diagnostic. Code and generated outputs are also available in the project repository. An archival version of the manuscript is available on Zenodo under DOI <https://doi.org/10.5281/zenodo.21870089>.

Disclosure of Interests. The author declares no competing interests relevant to the work.

A Prior-Work Boundary and Architecture Positioning

A.1 Relationship to earlier EVE publications

Table 4 maps established EVE results to the current experimental contribution.

Table 4. Relationship between earlier EVE publications and the current experimental analysis.

Earlier work	Main focus already established	Role in current analysis
Variational Distributional Neuron [41]	Local variational computational unit, local reference/KL structure, reparameterized state	Architecture is prior work; reused as the test object
Exploring the Dimensions of a Variational Neuron [42]	Latent dimensionality, local capacity, temporal persistence	k sweep retained as a consistency check
Measuring and Controlling Internal Activity in Variational Neural Units [68]	Internal KL/mean/activity diagnostics and controls	Diagnostics are prior work
The Neuron as a Latent State [69]	Frozen latent-state readout and multimodal forecasting	Readout concept established previously; current analysis uses different interventions
The Neuron Is the Distribution [70]	Distributional-neuron framing and controlled/network feasibility	General framing and EVE feasibility are prior work
Distributional Neurons: Making Uncertainty a Unit of Computation [71]	Unit-to-network/Transformer feasibility and probabilistic metrics	Composition and feasibility are prior work
July language-model reports [72,73,74,75]	Next-token prediction, internal activity, reliability/readout diagnostics	Language-model results remain separate from the controlled regression evidence
Current analysis	Weight-fixed interventions; controlled relocation of observation noise; fresh 50-seed exact-capacity output comparison; BNN/local-state object-location analysis	Experimental evidence evaluated in the controlled setting

A.2 Compact architecture crosswalk

Table 5. Representative neighboring constructions described by random object and computational location.

Architecture	Primary random object	Where it enters / is represented	Relation to EVE
Standard BNN	weights or functions conditioned on $\mathcal{D}$	throughout the network via posterior model/function uncertainty	different inferential object
VAE [27]	example-level latent z	shared decoder after an encoder	coarser latent granularity
Deep VIB [3]	stochastic bottleneck representation	representation bottleneck before predictor	representation-level rather than declared unit-level latent
Local-reparameterization BNN [26]	weight posterior	sampled pre-activations used as an estimator	local sampling, but weight-space inference
VIFO [63]	final-layer/output variable	predictive interface	output/final-layer location
Heteroscedastic regression [25]	observation variable / predicted scale	output head	main exact-capacity comparator
Gaussian-process neuron [76]	Gaussian-measure state defined by a GP unit	distribution is propagated analytically between GP neurons	neuron-level probabilistic computation with different object and propagation rule
EVE configuration	input-conditioned local latent z_i	sampled inside a declared hidden unit before downstream computation	intervention target in the controlled experiments

B Controlled Experiment Specification and Evaluation Protocol

This appendix specifies the data-generating process, optimization, architectures, statistical analysis, interventions, and detailed results required for reproducibility.

B.1 Data-generating process

For each split,

$$x \sim \mathcal{U}[-4, 4], \quad (6)$$

$$y \mid x \sim \mathcal{N}(m(x), \sigma^2(x)), \quad (7)$$

$$m(x) = 0.85 \tanh(1.35x) + 0.10x, \quad (8)$$

$$\sigma(x) = 0.07 + 0.52 \exp\left[-\frac{1}{2} \left(\frac{x}{0.85}\right)^2\right] + 0.05 (1 + \exp[-3(|x| - 2.7)])^{-1}. \quad (9)$$

The conditional mean is smooth, with a central high-variance region and mild edge variance. The controlled noise-relocation experiment replaces the center of

the main Gaussian variance component by -1.75 , 0 , or $+1.75$ while leaving the conditional mean unchanged.

B.2 Seeds, optimization, and evaluation

The mechanistic experiments use seeds $s \in \{0, \dots, 9\}$. The final exact-capacity Gaussian-affine configuration is selected using validation performance on development seeds $s \in \{40, \dots, 59\}$ and then evaluated on fresh paired seeds $s \in \{400, \dots, 449\}$ with no test-driven configuration changes. For every seed, independent training, validation, and test sets contain 2,500, 900, and 1,500 observations. Generator seeds are $10000+10s+1$, $10000+10s+2$, and $10000+10s+3$. Models are trained on CUDA-enabled GPUs using Adam with learning rate 2×10^{-3} and gradient clipping at norm 5. Training is capped at 1,200 full-batch epochs; validation is checked every 25 epochs, with early stopping after a minimum of 500 epochs and 12 checks without improvement.

The nonlinear $k = 2$ stochastic experiments use 32 Monte Carlo samples during training, 64 during validation, and 256 during predictive evaluation; their NLL uses a moment-matched Gaussian and CRPS is computed from predictive samples. The Gaussian-affine exact-capacity refinement has an analytic Gaussian marginal, so MSE, Gaussian NLL, Gaussian CRPS, and calibration are evaluated exactly without Monte Carlo scoring noise. Predictive calibration is evaluated at nominal coverages 50%, 60%, 70%, 80%, 90%, and 95%, summarized by mean absolute coverage error (MACE); 90% coverage and width are additionally reported as representative interval diagnostics.

B.3 Architectures

Principal nonlinear EVE variational neuron. The mechanistic EVE model has one scalar deterministic input state feeding a 1–32–32–4 encoder that parameterizes $(\mu, \log \sigma^2)$ for a diagonal-Gaussian latent with $k = 2$. A 3–32–32–1 decoder maps $[z, h]$ to the hidden activation. The reference distribution is $\mathcal{N}(0, I)$, $\beta = 10^{-3}$, and the latent-energy range penalty has coefficient 0.05 with target range $[0.01, 2.0]$. Encoder log-variance is clipped to $[-7, 3]$. The model has 2,472 trainable parameters.

Gaussian-affine EVE exact-capacity refinement. The fresh output-level control uses a separately declared $k = 1$ EVE configuration with deterministic trunk 1–42–53, a $53 \rightarrow 1$ deterministic mean head, and a $53 \rightarrow 1$ local scale head. It defines

$$q_\phi(z | h) = \mathcal{N}(0, s_\phi(h)^2), \quad a = m_\eta(h) + gz, \quad (10)$$

where $s_\phi(h) = \text{softplus}(r_\phi(h)) + 10^{-4}$, $g > 0$ is a learned scalar gain, the reference is $\mathcal{N}(0, 1)$, and $\beta = 10^{-3}$. Hence the declared stochastic unit activation has the analytic marginal $a | h \sim \mathcal{N}(m_\eta(h), g^2 s_\phi(h)^2)$. In this one-unit regression head, the scalar prediction is the identity readout of a , so the terminal unit is the declared location for the exact-capacity control. Predictive scale is carried by

the local latent state, and the fixed zero latent mean makes the mean-energy band unnecessary. The trainable count is $(1 \times 42 + 42) + (42 \times 53 + 53) + (53 + 1) + (53 + 1) + 1 = 2,472$. The final β setting is locked from validation seeds 40–59 before fresh seeds 400–449.

Auxiliary regularization sensitivity. A separate five-seed, fixed- $k = 2$, hidden-width-32 retraining-time control on the same synthetic family used 500 epochs per configuration and omitted the latent-energy band penalty, contrasting $\beta = 5 \times 10^{-2}$ with $\beta = 10^{-4}$. Mean local KL changes from 0.056 to 4.280 and mean latent energy $\|\mu_i\|^2$ from 0.029 to 7.909, while test MSE is 0.0776 versus 0.0779, CRPS 0.1275 versus 0.1278, and mean predictive standard deviation 0.2252 versus 0.2275. This retraining-time control characterizes sensitivity to β under the stated auxiliary configuration; the effect of changing the latent state with learned weights held fixed is addressed separately by the intervention protocol. The result shows that large β -driven changes in the internal variational regime can coexist with similar output metrics.

Capacity-matched deterministic hidden cell. The deterministic hidden-cell comparator uses a 1–32–32–4 encoder and a 5–32–32–1 decoder and has 2,536 trainable parameters. It uses a learned global Gaussian observation variance but has no local latent variable or local KL term.

Standard heteroscedastic output comparator. The standard output comparator has 1,158 parameters. Its hidden computation is deterministic; an input-dependent variance head predicts the observation scale at the output interface. This model serves as the preliminary output-location comparator, while the matched 2,472-parameter model below provides the exact-capacity comparison.

Exact-capacity heteroscedastic output comparator. The output model uses the same deterministic trunk 1–42–53, two output coordinates for predictive mean and raw scale, and one learned global raw-scale offset. Its count is $(1 \times 42 + 42) + (42 \times 53 + 53) + (53 \times 2 + 2) + 1 = 2,472$, exactly matching the Gaussian-affine EVE refinement.

B.4 Paired analysis

All headline effects are computed seed-wise before aggregation. For a metric M and paired models A and B ,

$$d_s = M_{A,s} - M_{B,s}, \quad \bar{d} = \frac{1}{n} \sum_{s=1}^n d_s. \quad (11)$$

Two-sided 95% Student- t intervals are reported for $\bar{d}$ as uncertainty summaries for the paired effects. The mechanistic experiments use $n = 10$; the fresh exact-capacity comparison uses $n = 50$ and additionally reports two-sided Wilcoxon signed-rank checks and deterministic paired-bootstrap intervals.

B.5 Intervention protocol with learned weights held fixed

For each seed, one $k = 2$ EVE variational neuron is trained once. Without modifying any learned weights, the test set is reevaluated under six policies: ordinary local-distribution sampling, local mean-state execution, standard-normal reference sampling, zero latent state, complete latent-state permutation across inputs, and input-conditioned variance permutation across inputs. Common latent and observation random numbers are reused wherever the policy permits. This design isolates downstream sensitivity to the latent mechanism at a fixed checkpoint.

B.6 Latent-dimensionality protocol

Latent dimensions $k = 1, 2, 4, 8$ are evaluated with hidden widths selected to keep capacity close to the $k = 2$ reference. The resulting parameter counts are 2,514, 2,472, 2,523, and 2,456. In addition to predictive metrics, the analysis records total KL, KL per latent dimension, latent mean energy, covariance trace, and a KL participation ratio computed from mean per-dimension KL contributions. The participation ratio serves as a descriptive allocation statistic, while task relevance is assessed separately through predictive metrics and interventions.

B.7 Controlled noise-relocation protocol

For each seed, three datasets reuse the same input samples and standardized Gaussian noise draws while moving only the center of the dominant conditional-noise component. The learned covariance trace is evaluated on a fixed input grid. The analysis reports correlation with the known conditional standard deviation, center of mass of learned versus true variance profiles, and the per-seed slope relating learned noise center to the manipulated true center. Circular shifts of the true variance profile are retained as a secondary descriptive null check alongside the center-tracking analysis.

B.8 Detailed controlled empirical results

Complete numerical results, confidence intervals, intervention definitions, and supporting figures follow for the controlled empirical suite.

B.9 Paired comparison with deterministic and output-probabilistic baselines

Table 6 reports seed-paired differences, defined as EVE minus comparator. Negative values favor EVE for MSE, NLL, and CRPS. The deterministic hidden-cell comparison is particularly informative about computational location: the deterministic cell has slightly more parameters (2,536 versus 2,472), yet the paired MSE difference has a confidence interval that includes zero while NLL and CRPS differ strongly.

Table 6. Ten-seed paired effects of the $k = 2$ EVE variational neuron. Differences are EVE minus comparator; negative values favor EVE for MSE, NLL, and CRPS.

Comparator	Metric	Paired Δ	95% Student- t CI
Capacity-matched deterministic hidden cell (2,536 parameters)	MSE	-0.000196	[-0.000534, 0.000142]
	NLL	-0.451	[-0.459, -0.443]
	CRPS	-0.0152	[-0.0162, -0.0143]
Standard heteroscedastic output model (1,158 parameters)	MSE	-0.00515	[-0.00616, -0.00414]
	NLL	-0.0323	[-0.0367, -0.0278]
	CRPS	-0.00372	[-0.00463, -0.00281]

Table 7. Fresh exact 2,472-parameter comparison over 50 paired seeds. Differences are Gaussian-affine EVE minus the heteroscedastic output model; lower is better for MSE, NLL, CRPS, and MACE.

Metric	Paired difference	95% CI	Interpretation
MSE	-0.0000046	[-0.0000331, 0.0000240]	CI includes zero
NLL	-0.000098	[-0.000494, 0.000298]	CI includes zero
CRPS	-0.0000095	[-0.0000394, 0.0000204]	CI includes zero
MACE (50–95%)	-0.000031	[-0.000443, 0.000381]	CI includes zero
Coverage 90%	+0.000480	[-0.000151, 0.001111]	CI includes zero
Width 90%	-0.000854	[-0.001654, -0.000054]	slightly narrower EVE intervals

The deterministic comparison shows that probabilistic scores can change even when the point-accuracy difference remains small and its confidence interval includes zero. The 1,158-parameter heteroscedastic model provides a preliminary output-level location comparison, and Section B.10 follows with the matched 2,472-versus-2,472 parameter control.

B.10 Exact-capacity output comparison

To control parameter count and predictive family in the output-level comparison, the fresh follow-up compares the standard shared-trunk heteroscedastic output regressor with the Gaussian-affine EVE configuration in Eq. (10). Both have exactly 2,472 trainable parameters and analytic Gaussian predictive marginals, but the random scale is represented at different computational locations: an output scale coordinate in the comparator versus an input-conditioned local latent state in EVE. The configuration is locked on validation seeds 40–59 and evaluated on fresh seeds 400–449.

This fresh control provides a sharper capacity- and predictive-family-matched comparison. With both parameter count and Gaussian predictive family matched, all four primary paired intervals include zero, and the Wilcoxon checks are concordant with those intervals. The result shows that a local variational state and an output-heteroscedastic parameterization can occupy different probabilistic locations while attaining no detected difference in the four primary predictive metrics on this controlled family.

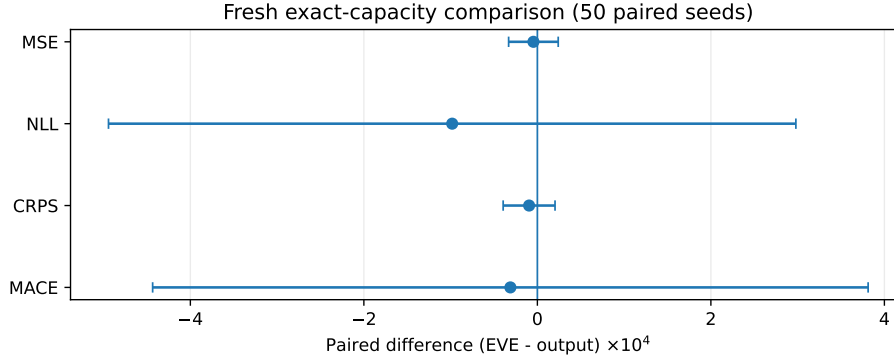


Fig. 1. Paired 95% confidence intervals for MSE, NLL, CRPS, and MACE in the fresh 50-seed exact 2,472-parameter comparison. All four intervals include zero.

Fresh-seed robustness check. The exact-capacity comparison uses 50 fresh paired seeds. Two-sided Wilcoxon signed-rank p -values are 0.848 for MSE, 0.383 for NLL, 0.466 for CRPS, and 0.947 for MACE, concordant with the paired Student- t intervals. EVE has the lower NLL on 30/50 seeds, while the aggregate paired CI spans zero and the Wilcoxon result is concordant with an unresolved aggregate difference in the primary metrics. The secondary predictive-standard-deviation difference is -0.000260 (95% CI $[-0.000503, -0.000017]$), corresponding to slightly sharper EVE predictions and reported as a descriptive sharpness diagnostic alongside the primary metrics.

B.11 Interventions with learned weights held fixed

To test whether the trained model uses the local state, each $k = 2$ EVE variational neuron is trained once and the *same trained weights* are evaluated under five latent-state interventions. Ordinary sampling from the learned input-conditioned distribution is the reference policy. Common random numbers are used wherever meaningful.

These interventions provide evidence of functional utilization and dependence on the local state with learned weights held fixed on this benchmark. They also separate two roles of the local distribution. The latent state as a whole carries information needed for point prediction, as shown by the large MSE increase after shuffling the complete state. The input-conditioned variance, in turn, changes NLL, CRPS, coverage, and interval width while the MSE change is small and its confidence interval includes zero. Its interpretation relative to known observation variability is examined separately through the controlled noise-relocation experiment.

Table 8. Ten-seed paired fixed-weight intervention effects for EVE relative to ordinary sampling from its learned local distribution. Positive ΔMSE , ΔNLL , and ΔCRPS indicate degradation.

Intervention	ΔMSE (95% CI)	ΔNLL (95% CI)	ΔCRPS (95% CI)
Use local mean instead of sampling	0.000114 [−0.000530, 0.000759]	2.82 [2.67, 2.97]	0.0186 [0.0174, 0.0197]
Sample from reference distribution	0.0273 [0.0208, 0.0339]	0.215 [0.158, 0.272]	0.0233 [0.0175, 0.0292]
Shuffle complete latent state across inputs	0.0994 [0.0631, 0.136]	0.713 [0.506, 0.920]	0.0769 [0.0504, 0.103]
Shuffle only input-conditioned latent variance	−0.000147 [−0.000340, 0.000046]	0.0842 [0.0297, 0.139]	0.00166 [0.000267, 0.00306]
Set latent state to zero	0.0196 [0.0151, 0.0241]	3.79 [3.48, 4.10]	0.0415 [0.0353, 0.0478]

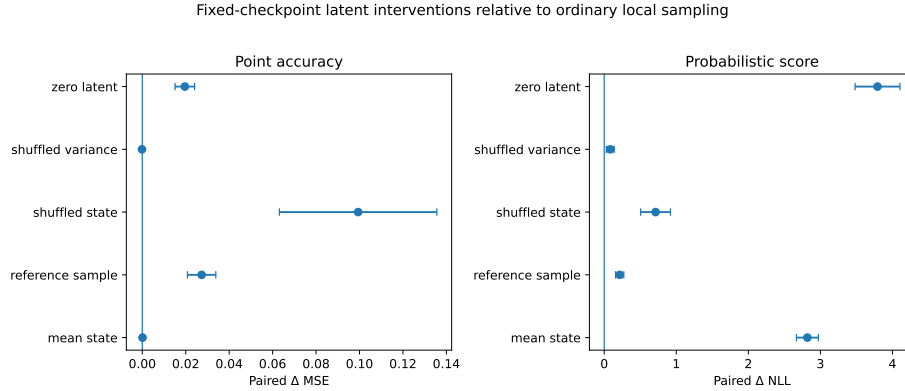


Fig. 2. Selected fixed-weight intervention effects. Shuffling the full latent state changes point accuracy. Shuffling only its input-conditioned variance leaves the MSE difference unresolved around zero but worsens NLL. Mean-state execution likewise leaves point accuracy unresolved around zero while strongly changing predictive dispersion.

B.12 Latent-dimensionality analysis

For EVE, the latent dimension k counts local latent degrees of freedom; network width counts deterministic units or features, and the declared unit partition remains fixed across the sweep. We evaluate $k \in \{1, 2, 4, 8\}$ while adjusting hidden width to keep parameter count within approximately 2.1% of the $k = 2$ model: 2,514, 2,472, 2,523, and 2,456 parameters respectively.

The paired MSE, NLL, and CRPS intervals include zero for $k = 1, 4, 8$ relative to $k = 2$, while total KL and the KL participation ratio change with k . The participation ratio is reported because it quantifies how local KL allocation is distributed across latent coordinates, an internal structural property complementary to predictive metrics. The combined record therefore separates latent-allocation changes from predictive effects. A future coordinate-by-coordinate in-

Table 9. Ten-seed paired differences relative to $k = 2$ under approximately matched capacity. Effective latent dimension is the participation ratio computed from per-dimension KL contributions.

k	Params	Δ MSE	Δ NLL	Δ effective latent dimension
1	2,514	-0.000116 [-0.000625, 0.000394]	0.00193 [-0.000846, 0.00470]	-0.749 [-0.894, -0.605]
2	2,472	reference	reference	reference
4	2,523	-0.000246 [-0.000627, 0.000135]	-0.000500 [-0.00316, 0.00216]	+1.17 [0.669, 1.66]
8	2,456	-0.000379 [-0.000989, 0.000230]	-0.00132 [-0.00417, 0.00153]	+3.25 [2.57, 3.93]

Table 10. Tracking of the manipulated conditional-noise structure over ten seeds. Correlations compare the learned local covariance trace with the known conditional standard deviation.

Noise location	Post-training correlation	95% CI
Left	0.747	[0.653, 0.842]
Center	0.982	[0.975, 0.989]
Right	0.793	[0.667, 0.918]

tervention could test which latent coordinates are individually important to the trained prediction.

B.13 Controlled relocation of conditional noise

To strengthen identification beyond a single fixed heteroscedastic pattern, we keep the conditional mean fixed while moving the high-variance region to the left, center, or right and test whether EVE’s learned local covariance follows that manipulation. For a given seed, the input samples and standardized Gaussian noise draws are held fixed across the three conditions; only the conditional noise scale changes.

Across the three manipulated conditions, the center of mass of the learned local covariance follows the true variance center with a mean per-seed tracking slope of 1.13 (95% CI [0.975, 1.29]). A slope of 1 corresponds to one-for-one tracking, so the interval includes exact one-for-one movement while the point estimate indicates modest overshoot. The correlation between true and learned centers across the three conditions is also consistently high across seeds (mean 0.992, 95% CI approximately [0.985, 0.998]). This manipulation provides benchmark-specific controlled evidence that the learned local covariance is responsive to controlled changes in observation variability under the known synthetic data-generating process.

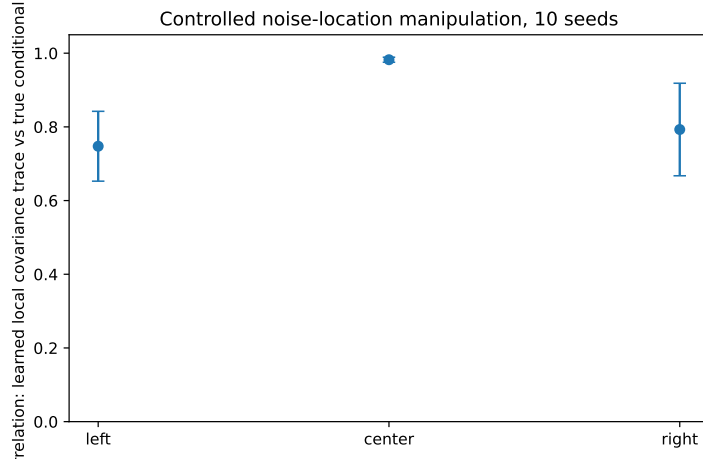


Fig. 3. Post-training correlation between learned local covariance trace and the known conditional standard deviation after moving the high-variance region left, center, and right.

B.14 Complete headline numerical results

Table 11. Fresh exact-capacity output comparison. Both models contain 2,472 trainable parameters. Values are 50-seed means with 95% confidence intervals for the model-level mean.

Metric	Heteroscedastic output	Gaussian-affine EVE
MSE	0.078282 [0.076700, 0.079864]	0.078277 [0.076697, 0.079858]
NLL	-0.300397 [-0.308316, -0.292478]	-0.300495 [-0.308503, -0.292487]
CRPS	0.127115 [0.125987, 0.128244]	0.127106 [0.125975, 0.128236]
Coverage 90%	0.897867 [0.894959, 0.900774]	0.898347 [0.895200, 0.901493]
Width 90%	0.736140 [0.730387, 0.741893]	0.735286 [0.729632, 0.740940]
MACE (50–95%)	0.011196 [0.009263, 0.013128]	0.011164 [0.009206, 0.013122]

The corresponding paired differences are reported in Table 7. Model-level intervals in Table 11 are descriptive; paired intervals are primary because both models share the same 50 fresh data splits. Calibration uses all six nominal levels (50, 60, 70, 80, 90, and 95%), summarized by MACE. The final $\beta = 10^{-3}$ setting was chosen from a validation-only candidate grid on seeds 40–59 and locked before seeds 400–449.

Table 12: Per-seed paired differences for the fresh exact-capacity comparison. Differences are Gaussian-affine EVE minus the heteroscedastic output model.

Seed	ΔMSE	ΔNLL	ΔCRPS	ΔMACE
400	+0.000193	-0.000366	+0.000039	+0.000222
401	+0.000006	-0.000194	-0.000013	+0.000333
402	-0.000064	+0.000778	+0.000022	-0.000333
403	+0.000027	+0.001074	+0.000082	-0.000444
404	-0.000021	-0.000775	-0.000052	+0.002778
405	+0.000023	+0.001668	+0.000159	+0.000333
406	-0.000110	-0.000592	-0.000105	+0.000778
407	-0.000137	-0.000093	-0.000096	+0.000111
408	+0.000001	+0.000822	+0.000016	-0.000333
409	-0.000000	-0.000214	+0.000046	-0.000556
410	+0.000052	-0.002185	-0.000036	-0.001444
411	-0.000117	-0.003570	-0.000297	+0.001889
412	-0.000055	-0.001276	-0.000054	-0.000111
413	-0.000048	-0.001464	-0.000062	+0.000778
414	+0.000044	+0.000431	+0.000021	-0.001000
415	+0.000009	-0.000247	+0.000002	-0.004333
416	+0.000049	+0.000230	+0.000007	+0.000111
417	+0.000159	+0.003420	+0.000227	+0.002111
418	-0.000170	-0.000870	-0.000125	+0.001111
419	+0.000039	-0.002721	-0.000102	-0.001333
420	+0.000030	+0.000230	+0.000023	+0.000889
421	-0.000021	+0.001490	+0.000081	+0.000556
422	+0.000063	-0.000224	+0.000011	-0.000444
423	-0.000169	+0.000882	-0.000036	-0.001222
424	-0.000046	-0.000392	-0.000030	-0.001444
425	+0.000027	-0.000162	+0.000009	+0.000333
426	-0.000032	-0.000057	-0.000037	-0.002333
427	+0.000020	+0.000417	+0.000049	+0.001444
428	-0.000146	-0.001260	-0.000089	+0.000222
429	+0.000087	-0.001152	-0.000001	-0.000667
430	+0.000124	-0.000506	+0.000001	+0.000556
431	-0.000240	-0.002513	-0.000282	+0.000889
432	+0.000152	+0.000048	+0.000069	-0.000222
433	-0.000032	+0.000052	-0.000059	-0.001667
434	-0.000028	+0.002484	-0.000004	-0.002333
435	+0.000080	-0.000533	+0.000058	+0.001889
436	+0.000129	+0.002152	+0.000241	+0.001222
437	-0.000135	-0.000531	-0.000105	-0.001111
438	+0.000097	+0.001141	+0.000097	+0.004111
439	+0.000004	-0.000256	-0.000004	+0.000000

Table 12 – continued

Seed	ΔMSE	ΔNLL	ΔCRPS	ΔMACE
440	-0.000046	+0.001980	-0.000049	-0.000667
441	-0.000037	-0.000664	+0.000016	+0.000000
442	+0.000023	+0.000114	+0.000017	+0.001222
443	+0.000086	+0.003016	+0.000174	+0.001667
444	-0.000038	-0.000668	-0.000075	-0.000000
445	+0.000223	+0.000655	+0.000133	-0.001333
446	-0.000031	-0.001745	-0.000056	+0.000333
447	-0.000247	-0.002141	-0.000245	-0.000667
448	-0.000010	-0.000324	-0.000024	-0.002667
449	+0.000005	-0.000299	-0.000042	-0.000778

Across the 50 fresh pairs, the two-sided Wilcoxon signed-rank p -values are 0.848 for MSE, 0.383 for NLL, 0.466 for CRPS, and 0.947 for MACE. The numbers of negative/positive paired differences are 24/26, 30/20, 26/24, and 24/26, respectively. The paired-bootstrap 95% intervals likewise include zero for all four primary metrics. The final reported configuration is evaluated on these fresh paired seeds.

Local-state diagnostics and comparison with the known synthetic truth. Across the 50 fresh EVE runs, mean local KL is 0.505 ± 0.032 , mean q -scale is 1.005 ± 0.026 , learned gain is 0.2225 ± 0.0030 , and mean effective predictive standard deviation is 0.22351 ± 0.00605 . On the ten development seeds 40–49, where the true synthetic mean and variance are known exactly, mean-function RMSE is 0.01597 for EVE versus 0.01617 for the output model, variance RMSE is 0.00976 versus 0.01122, and variance correlation is 0.99764 versus 0.99663. These quantities are reported as development-set diagnostics against the known synthetic truth.

Table 13. Controlled-noise tracking summaries over ten seeds.

Noise location	Pre-training correlation	Post-training correlation	Absolute center error
Left	-0.464 [-0.787, -0.141]	0.747 [0.653, 0.842]	0.379 [0.146, 0.612]
Center	0.150 [-0.086, 0.386]	0.982 [0.975, 0.989]	0.225 [0.128, 0.322]
Right	0.425 [0.052, 0.798]	0.793 [0.667, 0.918]	0.261 [0.045, 0.476]

The mean tracking slope across seeds is 1.13 with 95% CI [0.975, 1.29]. Unit slope lies within the interval, while the point estimate indicates modest overshoot. Together with the condition-wise correlations, this provides controlled evidence that the learned local covariance follows the manipulated data-generating variance pattern. The experiment therefore supplies the controlled evidence that the local covariance responds to this known synthetic variance manipulation.

Algorithm 1 Generic local variational computation.

Require: h_i, c_i

- 1: obtain μ_i, σ_i from $q_{\phi_i}(z_i | h_i, c_i)$
 - 2: sample $\epsilon_i \sim \mathcal{N}(0, I)$ and set $z_i \leftarrow \mu_i + \sigma_i \odot \epsilon_i$
 - 3: compute $a_i \leftarrow F_{\eta_i}(h_i, z_i)$
 - 4: compute $\mathcal{K}_i \leftarrow \text{KL}(q_{\phi_i} \| p_{\psi_i})$
 - 5: **return** $(a_i, \mathcal{K}_i)$
-

C Supplementary Formalism

Two formal details support the probabilistic interpretation: the separation of weight and observation uncertainty, and the generic local variational computation.

C.1 Weight uncertainty and observation uncertainty

In heteroscedastic regression with uncertain weights,

$$y | x, w \sim \mathcal{N}(\mu_w(x), \sigma_w^2(x)), \quad (12)$$

the law of total variance separates variation in the observation model from variation induced by the weight posterior:

$$\text{Var}(y_* | x_*, \mathcal{D}) = \mathbb{E}_{p(w|\mathcal{D})}[\text{Var}(y_* | x_*, w)] \quad (13)$$

$$+ \text{Var}_{p(w|\mathcal{D})}(\mathbb{E}[y_* | x_*, w]). \quad (14)$$

This decomposition separates an input-conditioned local latent state from a BNN weight posterior as distinct random objects [20,25].

C.2 Generic variational-neuron computation

References

1. Abdar, M., Pourpanah, F., Hussain, S., Rezazadegan, D., Liu, L., Ghavamzadeh, M., et al.: A review of uncertainty quantification in deep learning: Techniques, applications and challenges. *Information Fusion* **76**, 243–297 (2021). <https://doi.org/10.1016/j.inffus.2021.05.008>
2. Achille, A., Soatto, S.: Information dropout: Learning optimal representations through noisy computation. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **40**(12), 2897–2905 (2018). <https://doi.org/10.1109/TPAMI.2017.2784440>
3. Alemi, A.A., Fischer, I., Dillon, J.V., Murphy, K.: Deep variational information bottleneck. In: *International Conference on Learning Representations* (2017)
4. Amini, A., Schwarting, W., Soleimany, A.P., Rus, D.: Deep evidential regression. In: *Advances in Neural Information Processing Systems*. vol. 33 (2020)

5. Arbel, J., Pitas, K., Vladimirova, M., Fortuin, V.: A primer on Bayesian neural networks: Review and debates. *Statistical Science* **41**(2), 316–353 (28 May 2026). <https://doi.org/10.1214/24-ST5969>
6. Bellemare, M.G., Dabney, W., Munos, R.: A distributional perspective on reinforcement learning. In: *Proceedings of the 34th International Conference on Machine Learning*. *Proceedings of Machine Learning Research*, vol. 70, pp. 449–458. PMLR (2017)
7. Bengio, Y., Léonard, N., Courville, A.: Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432* (2013). <https://doi.org/10.48550/arXiv.1308.3432>
8. Bishop, C.M.: Mixture density networks. Tech. Rep. NCRG/94/004, Aston University (1994)
9. Bishop, C.M.: *Pattern Recognition and Machine Learning*. Springer, New York (2006)
10. Blei, D.M., Kucukelbir, A., McAuliffe, J.D.: Variational inference: A review for statisticians. *Journal of the American Statistical Association* **112**(518), 859–877 (2017). <https://doi.org/10.1080/01621459.2017.1285773>
11. Blundell, C., Cornebise, J., Kavukcuoglu, K., Wierstra, D.: Weight uncertainty in neural networks. In: *Proceedings of the 32nd International Conference on Machine Learning*. *Proceedings of Machine Learning Research*, vol. 37, pp. 1613–1622. PMLR (2015)
12. Burda, Y., Grosse, R., Salakhutdinov, R.: Importance weighted autoencoders. In: *International Conference on Learning Representations* (2016)
13. Chung, J., Kastner, K., Dinh, L., Goel, K., Courville, A.C., Bengio, Y.: A recurrent latent variable model for sequential data. In: *Advances in Neural Information Processing Systems*. vol. 28 (2015)
14. Depeweg, S., Hernández-Lobato, J.M., Doshi-Velez, F., Udluft, S.: Decomposition of uncertainty in Bayesian deep learning for efficient and risk-sensitive learning. In: *Proceedings of the 35th International Conference on Machine Learning*. *Proceedings of Machine Learning Research*, vol. 80, pp. 1184–1193. PMLR (2018)
15. Fortuin, V.: Priors in Bayesian deep learning: A review. *International Statistical Review* **90**(3), 563–591 (2022). <https://doi.org/10.1111/insr.12502>
16. Gal, Y., Ghahramani, Z.: Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. In: *Proceedings of the 33rd International Conference on Machine Learning*. *Proceedings of Machine Learning Research*, vol. 48, pp. 1050–1059. PMLR (2016)
17. Gawlikowski, J., Tassi, C.R.N., Ali, M., Lee, J., Humt, M., Feng, J., et al.: A survey of uncertainty in deep neural networks. *Artificial Intelligence Review* **56**(Suppl 1), 1513–1589 (2023). <https://doi.org/10.1007/s10462-023-10562-9>
18. Graves, A.: Practical variational inference for neural networks. In: *Advances in Neural Information Processing Systems*. vol. 24 (2011)
19. Hernández-Lobato, J.M., Adams, R.P.: Probabilistic backpropagation for scalable learning of Bayesian neural networks. In: *Proceedings of the 32nd International Conference on Machine Learning*. *Proceedings of Machine Learning Research*, vol. 37, pp. 1861–1869. PMLR (2015)
20. Hüllermeier, E., Waegeman, W.: Aleatoric and epistemic uncertainty in machine learning: An introduction to concepts and methods. *Machine Learning* **110**(3), 457–506 (2021). <https://doi.org/10.1007/s10994-021-05946-3>
21. Izmailov, P., Nicholson, P., Lotfi, S., Wilson, A.G.: Dangers of Bayesian model averaging under covariate shift. In: *Advances in Neural Information Processing Systems*. vol. 34 (2021)

22. Jacobs, R.A., Jordan, M.I., Nowlan, S.J., Hinton, G.E.: Adaptive mixtures of local experts. *Neural Computation* **3**(1), 79–87 (1991). <https://doi.org/10.1162/neco.1991.3.1.79>
23. Jang, E., Gu, S., Poole, B.: Categorical reparameterization with Gumbel–Softmax. In: *International Conference on Learning Representations* (2017)
24. Jospin, L.V., Laga, H., Boussaid, F., Buntine, W., Bennamoun, M.: Hands-on Bayesian neural networks—a tutorial for deep learning users. *IEEE Computational Intelligence Magazine* **17**(2), 29–48 (2022). <https://doi.org/10.1109/MCI.2022.3155327>
25. Kendall, A., Gal, Y.: What uncertainties do we need in Bayesian deep learning for computer vision? In: *Advances in Neural Information Processing Systems*. vol. 30 (2017)
26. Kingma, D.P., Salimans, T., Welling, M.: Variational dropout and the local reparameterization trick. In: *Advances in Neural Information Processing Systems*. vol. 28 (2015)
27. Kingma, D.P., Welling, M.: Auto-encoding variational Bayes. In: *International Conference on Learning Representations* (2014). <https://doi.org/10.48550/arXiv.1312.6114>
28. Lakshminarayanan, B., Pritzel, A., Blundell, C.: Simple and scalable predictive uncertainty estimation using deep ensembles. In: *Advances in Neural Information Processing Systems*. vol. 30 (2017)
29. MacKay, D.J.C.: A practical Bayesian framework for backpropagation networks. *Neural Computation* **4**(3), 448–472 (1992). <https://doi.org/10.1162/neco.1992.4.3.448>
30. Maddison, C.J., Mnih, A., Teh, Y.W.: The concrete distribution: A continuous relaxation of discrete random variables. In: *International Conference on Learning Representations* (2017)
31. Maddox, W.J., Garipov, T., Izmailov, P., Vetrov, D., Wilson, A.G.: A simple baseline for Bayesian uncertainty in deep learning. In: *Advances in Neural Information Processing Systems*. vol. 32 (2019)
32. Malinin, A., Gales, M.: Predictive uncertainty estimation via prior networks. In: *Advances in Neural Information Processing Systems*. vol. 31 (2018)
33. Neal, R.M.: *Bayesian Learning for Neural Networks*, Lecture Notes in Statistics, vol. 118. Springer, New York (1996). <https://doi.org/10.1007/978-1-4612-0745-0>
34. Oh, S.J., Murphy, K.P., Pan, J., Roth, J., Schroff, F., Gallagher, A.C.: Modeling uncertainty with hedged instance embeddings. In: *International Conference on Learning Representations* (2019)
35. Ovadia, Y., Fertig, E., Ren, J., Nado, Z., Sculley, D., Nowozin, S., et al.: Can you trust your model’s uncertainty? evaluating predictive uncertainty under dataset shift. In: *Advances in Neural Information Processing Systems*. vol. 32 (2019)
36. Ranganath, R., Tang, L., Charlin, L., Blei, D.M.: Deep exponential families. In: *Proceedings of the Eighteenth International Conference on Artificial Intelligence and Statistics*. *Proceedings of Machine Learning Research*, vol. 38, pp. 762–771 (2015)
37. Rasmussen, C.E., Williams, C.K.I.: *Gaussian Processes for Machine Learning*. MIT Press, Cambridge, MA (2006)
38. Rezende, D.J., Mohamed, S.: Variational inference with normalizing flows. In: *Proceedings of the 32nd International Conference on Machine Learning*. *Proceedings of Machine Learning Research*, vol. 37, pp. 1530–1538. PMLR (2015)

39. Rezende, D.J., Mohamed, S., Wierstra, D.: Stochastic backpropagation and approximate inference in deep generative models. In: Proceedings of the 31st International Conference on Machine Learning. Proceedings of Machine Learning Research, vol. 32, pp. 1278–1286. PMLR (2014)
40. Ritter, H., Botev, A., Barber, D.: Online structured Laplace approximations for overcoming catastrophic forgetting. In: Advances in Neural Information Processing Systems. vol. 31 (2018)
41. Ruffenach, Y.: Variational distributional neuron. arXiv preprint arXiv:2602.18250v1 (20 Feb 2026). <https://doi.org/10.48550/arXiv.2602.18250>
42. Ruffenach, Y.: Exploring the dimensions of a variational neuron. arXiv preprint arXiv:2603.13849v1 (14 Mar 2026). <https://doi.org/10.48550/arXiv.2603.13849>
43. Sensoy, M., Kaplan, L., Kandemir, M.: Evidential deep learning to quantify classification uncertainty. In: Advances in Neural Information Processing Systems. vol. 31 (2018)
44. Shi, Y., Jain, A.K.: Probabilistic face embeddings. In: Proceedings of the IEEE/CVF International Conference on Computer Vision. pp. 6902–6911 (2019)
45. Sønderby, C.K., Raiko, T., Maaløe, L., Sønderby, S.K., Winther, O.: Ladder variational autoencoders. In: Advances in Neural Information Processing Systems. vol. 29 (2016)
46. Tang, C., Salakhutdinov, R.R.: Learning stochastic feedforward neural networks. In: Advances in Neural Information Processing Systems. vol. 26 (2013)
47. Welling, M., Teh, Y.W.: Bayesian learning via stochastic gradient Langevin dynamics. In: Proceedings of the 28th International Conference on Machine Learning. pp. 681–688 (2011)
48. Williams, R.J.: Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning* **8**, 229–256 (1992). <https://doi.org/10.1007/BF00992696>
49. Wilson, A.G., Izmailov, P.: Bayesian deep learning and a probabilistic perspective of generalization. In: Advances in Neural Information Processing Systems. vol. 33, pp. 4697–4708 (2020)
50. Bayer, J., Osendorfer, C., Urban, S., van der Smagt, P.: Training neural networks with implicit variance. In: Neural Information Processing, LNCS, vol. 8227, pp. 132–139. Springer (2013). https://doi.org/10.1007/978-3-642-42042-9_17
51. Bui, L.M., Tran Huu, T., Dinh, D., Nguyen, T.M., Hoang, T.N.: Revisiting kernel attention with correlated Gaussian process representation. In: Proceedings of the Fortieth Conference on Uncertainty in Artificial Intelligence. PMLR 244, 450–470 (2024). <https://proceedings.mlr.press/v244/bui24a.html>
52. Diamzon, J., Venturi, D.: Uncertainty propagation in feed-forward neural network models. *Neural Networks* **194**, 108178 (Feb 2026). <https://doi.org/10.1016/j.neunet.2025.108178>
53. Gast, J., Roth, S.: Lightweight probabilistic deep networks. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 3369–3378 (2018).
54. Hu, J., Yi, X., Li, W., Sun, M., Xie, X.: Fuse it more deeply! A variational Transformer with layer-wise latent variable inference for text generation. In: NAACL-HLT, pp. 697–716 (2022). <https://doi.org/10.18653/v1/2022.naacl-main.51>
55. Kingma, D.P., Welling, M.: Efficient gradient-based inference through transformations between Bayes nets and neural nets. In: Proceedings of the 31st Inter-

- national Conference on Machine Learning. PMLR 32, 1782–1790 (2014). <https://proceedings.mlr.press/v32/kingma14.html>
56. Kofnov, A., Kapla, D., Bartocci, E., Bura, E.: Exact upper and lower bounds for the output distribution of neural networks with random inputs. In: Proceedings of the 42nd International Conference on Machine Learning. PMLR 267, 31133–31157 (2025). <https://proceedings.mlr.press/v267/kofnov25a.html>
 57. Luis, C.E., Bottero, A.G., Vinogradska, J., Berkenkamp, F., Peters, J.: Uncertainty representations in state-space layers for deep reinforcement learning under partial observability. *Transactions on Machine Learning Research*, 1–31 (2025). <https://openreview.net/forum?id=rFPnsOWJyg>
 58. Monchot, P., Coquelin, L., Petit, S.J., Marmin, S., Le Pennec, E., Fischer, N.: Input uncertainty propagation through trained neural networks. In: Proceedings of the 40th International Conference on Machine Learning. PMLR 202, 25140–25173 (2023). <https://proceedings.mlr.press/v202/monchot23a.html>
 59. Nguyen, T.T., Choi, J.: Markov information bottleneck to improve information flow in stochastic neural networks. *Entropy* **21**(10), 976 (2019). <https://doi.org/10.3390/e21100976>
 60. Sankararaman, K.A., Wang, S., Fang, H.: BayesFormer: Transformer with uncertainty estimation. *arXiv:2206.00826* (2022). <https://doi.org/10.48550/arXiv.2206.00826>
 61. Shekhovtsov, A., Flach, B., Busta, M.: Feed-forward uncertainty propagation in belief and neural networks. *arXiv:1803.10590* (2018). <https://doi.org/10.48550/arXiv.1803.10590>
 62. Wang, H., Shi, X., Yeung, D.Y.: Natural-parameter networks: A class of probabilistic neural networks. In: *Advances in Neural Information Processing Systems*, vol. 29, pp. 118–126 (2016).
 63. Wei, Y., Khardon, R.: Variational inference on the final-layer output of neural networks. *Transactions on Machine Learning Research* (2024). <https://openreview.net/forum?id=mT0zXLmLKr>
 64. Wright, O., Nakahira, Y., Moura, J.M.F.: An analytic solution to covariance propagation in neural networks. In: *Proceedings of the 27th International Conference on Artificial Intelligence and Statistics*. PMLR 238, 4087–4095 (2024). <https://proceedings.mlr.press/v238/wright24a.html>
 65. Xue, B., Yu, J., Xu, J., Liu, S., Hu, S., Ye, Z., et al.: Bayesian Transformer language models for speech recognition. In: *2021 IEEE International Conference on Acoustics, Speech and Signal Processing*, pp. 7378–7382 (2021). <https://doi.org/10.1109/ICASSP39728.2021.9414046>
 66. Bahuleyan, H., Mou, L., Vechtomova, O., Poupart, P.: Variational attention for sequence-to-sequence models. In: *Proceedings of the 27th International Conference on Computational Linguistics*, pp. 1672–1682. Association for Computational Linguistics, Santa Fe, New Mexico, USA (2018). <https://aclanthology.org/C18-1142/>
 67. Dhuliawala, S.Z., Sachan, M., Allen, C.: Variational classification: A probabilistic generalization of the softmax classifier. *Transactions on Machine Learning Research* (2024). <https://openreview.net/forum?id=EWv9XG0pB3>
 68. Ruffenach, Y.: Measuring and controlling internal activity in variational neural units. *Research Square preprint* (4 May 2026). <https://doi.org/10.21203/rs.3.rs-9585590/v1>
 69. Ruffenach, Y.: The neuron as a latent state: Classical variational readout in distributional neural units. *Research Square preprint* (30 Jun 2026). <https://doi.org/10.21203/rs.3.rs-10177882/v1>

70. Ruffenach, Y.: The neuron is the distribution. Research Square preprint (14 Jul 2026). <https://doi.org/10.21203/rs.3.rs-10333402/v1>
71. Ruffenach, Y.: Distributional neurons: Making uncertainty a unit of computation. Research Square preprint (21 Jul 2026). <https://doi.org/10.21203/rs.3.rs-10401116/v1>
72. Ruffenach, Y.: Learning distributions inside a language model: Variational neurons for measurable internal uncertainty and reliability monitoring. Zenodo preprint (27 Jul 2026). <https://doi.org/10.5281/zenodo.21632472>
73. Ruffenach, Y.: Measuring local posterior activity in variational language model units. Zenodo preprint (28 Jul 2026). <https://doi.org/10.5281/zenodo.21641827>
74. Ruffenach, Y.: Variational distributional neurons for measurable internal uncertainty in language models. Zenodo preprint (29 Jul 2026). <https://doi.org/10.5281/zenodo.21669216>
75. Ruffenach, Y.: Measuring internal probabilistic activity with variational distributional neurons. Zenodo preprint (29 Jul 2026). <https://doi.org/10.5281/zenodo.21668942>
76. Meng, Q., Zhang, Y.: Exact uncertainty propagation via Gaussian process neurons. In: Proceedings of the 42nd Conference on Uncertainty in Artificial Intelligence. Proceedings of Machine Learning Research, vol. 337, pp. 4497–4513 (2026). <https://proceedings.mlr.press/v337/meng26a.html>